

Documentação Técnica - Energift

BRUNO GUILHERME DE JESUS FERNANDES – RM 559955

GABRIEL DOS SANTOS MELO – RM 560346

VINICIUS TOLEDO BATISTA – RM 559979

NICOLAS MEDEIROS MOREIRA KUBITZA – RM 559413

IZABELE ARAÚJO VITTE - RM559971

Informações Gerais

Título do Projeto: Energift

Nome dos Integrantes: [Removido]

Descrição do Pipeline CI/CD

Ferramenta Utilizada: GitHub Actions

O pipeline de CI/CD foi implementado utilizando GitHub Actions para automatizar o processo de build, teste e deploy da aplicação Energift. Ele é configurado para ser acionado automaticamente em eventos de push e pull_request nas branches main e develop.

Etapas e Lógica

O pipeline é dividido em jobs principais, agora com deploy real para o Azure App Service:

1. build-and-test:

- Checkout do Código: Clona o repositório para o ambiente do runner.
- Setup.NET: Instala a versão 8.0.x do .NET SDK.
- Restauração de Dependências: Executa dotnet restore para baixar todas as dependências do projeto.
- Build da Aplicação: Compila a solução (Energift.Fiap.sln) em modo Release.
- Execução de Testes: Executa os testes unitários do projeto Energift.Tests e gera um arquivo de resultados (test-results.trx).
- Upload de Resultados de Teste: Os resultados dos testes são salvos como um artefato para análise posterior.
- Publicação da Aplicação: Publica a aplicação (src/Energift.Fiap.csproj) em modo Release.
- Upload do Artefato de Build: O pacote da aplicação publicada é salvo como um artefato.

2. build-and-push-docker:

- Dependência: Este job só é executado após a conclusão bem-sucedida do build-and-test.
- Download do Artefato: Baixa o artefato de build gerado na etapa anterior.
- Login no GitHub Container Registry (GHCR): Autentica no GHCR usando o GITHUB_TOKEN (fornecido automaticamente pelo GitHub Actions).
- Build e Push da Imagem Docker: Constrói a imagem Docker da aplicação e a envia para o GHCR. A tag da imagem é baseada no github.sha.

3. deploy-staging:

- Dependência: Este job só é executado após a conclusão bem-sucedida do build-and-push-docker.
- Login no Azure: Autentica no Azure usando as AZURE_CREDENTIALS (Service Principal) configuradas como secrets.
- Deploy para Azure Web App (Staging): Implanta a imagem Docker mais recente do GHCR no Azure App Service configurado para o ambiente de Staging (AZURE_WEBAPP_NAME_STAGING).

4. deploy-production:

- Dependência: Este job só é executado após a conclusão bem-sucedida do deploy-staging.
- Login no Azure: Autentica no Azure.
- Deploy para Azure Web App (Production): Implanta a imagem Docker mais recente do GHCR no Azure App Service configurado para o ambiente de Produção (AZURE_WEBAPP_NAME_PRODUCTION).

Docker: Arquitetura, Comandos e Imagem Criada

Arquitetura de Containerização

A aplicação Energift é containerizada utilizando Docker, o que garante um ambiente consistente para desenvolvimento, testes e produção. A arquitetura envolve:

- Imagem da Aplicação: Uma imagem Docker personalizada para a API C#.NET 8.0.
- Banco de Dados: Um contêiner separado para o PostgreSQL, garantindo isolamento e persistência de dados.
- Orquestração: Docker Compose para gerenciar e interligar os contêineres da aplicação e do banco de dados.

Dockerfile

O Dockerfile é multi-estágio, o que otimiza o tamanho final da imagem e separa as etapas de build e runtime. As estratégias adotadas são:

- build-env: Utiliza `mcr.microsoft.com/dotnet/sdk:8.0` para compilar a aplicação. Copia apenas o necessário para restaurar dependências e depois o restante do código. Realiza o `dotnet restore` e `dotnet build`.
- test-env: Um estágio dedicado para a execução dos testes unitários, garantindo que os testes sejam executados antes da publicação da aplicação. Os resultados são salvos em um diretório específico.
- publish: Publica a aplicação (`dotnet publish`) para gerar os artefatos de deploy.
- final: Utiliza `mcr.microsoft.com/dotnet/aspnet:8.0` como imagem base, que é menor e contém apenas o runtime necessário. Copia os artefatos publicados do estágio publish para esta imagem. Define o `ENTRYPOINT` para iniciar a aplicação.

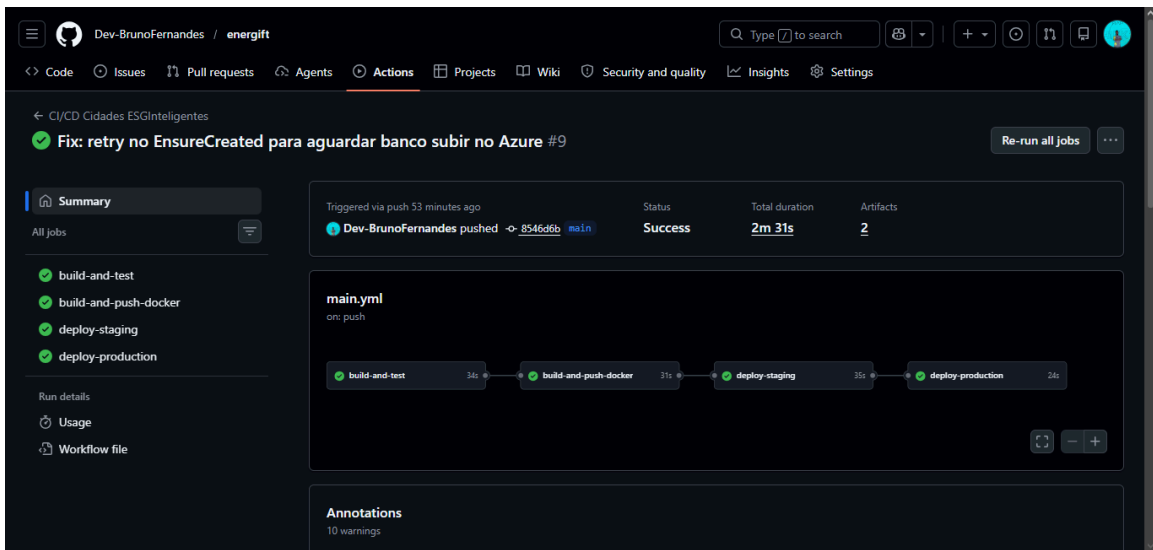
Comandos Docker Essenciais

- Construir a imagem da aplicação:
`docker build -t cidades-esginteligentes-api .`
- Executar a aplicação (sem Docker Compose):
`docker run -p 8080:8080 cidades-esginteligentes-api`
- Subir a aplicação e o banco de dados com Docker Compose:
`docker-compose up --build`
- Parar os serviços do Docker Compose:
`docker-compose down`

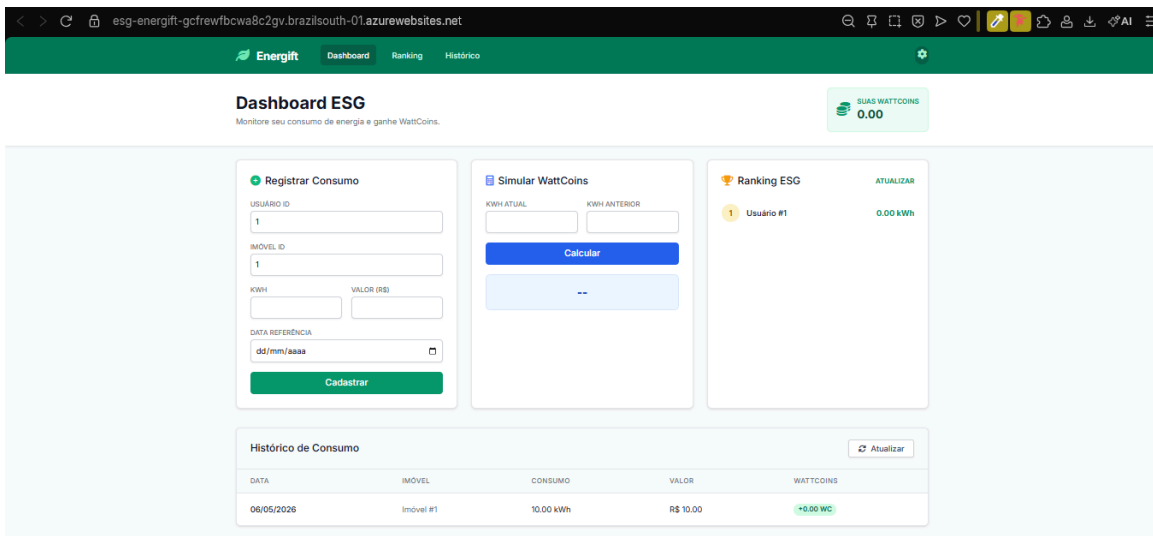
Imagem Criada

A imagem Docker final contém a aplicação Energift pronta para ser executada, com todas as suas dependências. Ela é otimizada para ser leve e eficiente.

Prints do Pipeline Rodando (Build, Testes, Deploy)



Prints dos Ambientes Staging e Produção Funcionando



Desafios Encontrados e Como Foram Resolvidos

1. Erro de dotnet: command not found:

- Desafio: Durante a configuração inicial, o comando dotnet não estava disponível no ambiente do sandbox, impedindo a criação do projeto de testes.
- Resolução: Foi necessário instalar o .NET SDK 8.0 no ambiente do sandbox utilizando sudo apt-get install dotnet-sdk-8.0.

2. Erros de Referência e Compilação de Testes:

- Desafio: Inicialmente, o projeto de testes foi criado dentro da pasta src e referenciado de forma incorreta, causando erros de compilação e referências circulares. O projeto principal tentava compilar os arquivos de teste, e as bibliotecas xUnit não eram reconhecidas.
- Resolução:
 - O projeto de testes (Energift.Tests) foi movido para a raiz do projeto (/home/ubuntu/Cidades_ESGInteligentes/) para isolá-lo do código-fonte principal.
 - As dependências do xUnit (xunit, xunit.runner.visualstudio, Microsoft.NET.Test.Sdk) foram adicionadas explicitamente ao projeto de testes.
 - Foi realizada uma limpeza (dotnet clean) e reconstrução (dotnet build) da solução para garantir que as alterações fossem aplicadas corretamente.

3. Otimização do Dockerfile para CI/CD

- Desafio: O Dockerfile inicial não estava otimizado para um pipeline de CI/CD, com etapas de build e teste não claramente definidas.
- Resolução: O Dockerfile foi reestruturado para usar um build multi-estágio, incluindo estágios dedicados para build-env, test-env, publish e final. Isso permite que os testes sejam executados durante o processo de build da imagem e resulta em uma imagem final menor e mais segura.

4. Erro de Arquivo .sln não encontrado no Dockerfile:

- Desafio: Após mover o arquivo .sln para a raiz do projeto, o Dockerfile não conseguia encontrá-lo devido a caminhos relativos incorretos no contexto de build.
- Resolução: Ajustei o Dockerfile para copiar o sln da raiz e os .csproj de suas respectivas pastas (src/ e Energift.Tests/). Além disso, o comando dotnet publish foi ajustado para não usar --no-build, garantindo que os artefatos fossem gerados corretamente.

5. Erro de Referência do Projeto de Testes no VS Code:

- Desafio: Ao abrir o projeto no VS Code, o arquivo .sln (que estava em src/) não conseguia encontrar o projeto de testes (Energift.Tests) que havia sido movido para a raiz.
- Resolução: Movi o arquivo Energift.Fiap.sln para a raiz do projeto e recriei-o usando dotnet new sln e dotnet sln add para garantir que todas as referências estivessem corretas e relativas à raiz do projeto.

6. Erro de Conflito de Contêiner Docker:

- Desafio: Ao tentar subir o docker-compose, um erro de conflito indicava que um contêiner com o mesmo nome já existia, mesmo que parado.
- Resolução: Foi necessário remover explicitamente o contêiner em conflito usando docker

rm -f postgres-db e docker rm -f energift-api antes de tentar subir o docker-compose novamente.

7. Restrição de Azure Container Registry (ACR) em Conta de Estudante:

- **Desafio:** Contas de estudante podem ter restrições de acesso a serviços pagos como o Azure Container Registry, impedindo o push de imagens Docker para um registro privado no Azure.
- **Resolução:** O pipeline de CI/CD foi adaptado para utilizar o GitHub Container Registry (GHCR). O GHCR é gratuito, integrado ao GitHub e permite o armazenamento de imagens Docker diretamente no repositório, que podem ser puxadas pelo Azure App Service para Containers.

Checklist de Entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✓
Dockerfile funcional	✓
docker-compose.yml ou arquivos Kubernetes	✓
Pipeline com etapas de build, teste e deploy	✓
README.md com instruções e prints	✓
Documentação técnica com evidências (PDF ou PPT)	✓
Deploy realizado nos ambientes staging e produção	✓